Job Aid — Hand-Testing the Alliance API Fetchers

This walks through validating the API-fetcher integration on the `wire-api-sources` branch by building a candidate Docker image and running each step manually. Every step has a check so you can stop and inspect before committing to the next one.

If a candidate run completes successfully end-to-end, that image **is** the new production image — promote with `docker tag`, no rebuild.

> Run from the AllianceMineDev EC2 host (172.31.60.197). Paths assume `~/agr_intermine_builder` is the repo root.

Prerequisites

```
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.60.197
cd ~/agr_intermine_builder
git fetch && git checkout wire-api-sources && git pull
```

Confirm the branch is current:

```
git log --oneline -1
# Expect: 2918a38 Wire Alliance API fetchers into extract_data
```

Make sure `.env` is in place and has RDS credentials filled in. If you copied an old `.env` from the previous build, **add these two lines** before continuing:

```
cd ~/agr_intermine_builder/docker/alliancemine
grep -q '^IMAGE_TAG=' .env || echo 'IMAGE_TAG=9.0.0-api' >> .env
grep -q '^BIO_SOURCES_BRANCH=' .env || echo 'BIO_SOURCES_BRANCH=wire-api-sources' >> .env
cat .env | grep -E 'IMAGE_TAG|BIO_SOURCES_BRANCH'
```

Expected output:

```
IMAGE_TAG=9.0.0-api
BIO_SOURCES_BRANCH=wire-api-sources
```

The two settings together mean: build a candidate image tagged `alliancemine-builder:9.0.0-api` from the bio-sources `wire-api-sources` branch, leaving `alliancemine-builder:latest` (the current production image) untouched.

Step 1 — Build the candidate image

```
cd ~/agr_intermine_builder/docker/alliancemine
docker compose build --no-cache alliancemine-builder
```

`--no-cache` is required because the Dockerfile clones bio-sources during the build; without it, Docker will reuse the cached clone of `master` and ignore the branch override.

Wall time: ~5 min.

Check 1.1 — Image exists with the right tag

```
docker images | grep alliancemine-builder
```

Expected: at least one row showing `alliancemine-builder` with tag `9.0.0-api`. If you also have a `latest` row, that's the existing production image — leave it alone.

Check 1.2 — Bio-sources branch is the right one

```
docker run --rm --entrypoint /bin/bash alliancemine-builder:9.0.0-api -c \
  'cd /root/alliancemine-bio-sources && git branch --show-current && git log --oneline -1'
```

Expected:

```
wire-api-sources
27eafa8 wire new Phase-5 fetchers into fetch_all.py and refresh docs
```

If you see `master` instead, the build cached an old clone — re-run Step 1 with `--no-cache` and confirm `BIO_SOURCES_BRANCH` is in `.env`.

Check 1.3 — Fetcher scripts are present

```
docker run --rm --entrypoint /bin/bash alliancemine-builder:9.0.0-api -c \
  'ls /root/alliancemine-bio-sources/scripts/'
```

Expected files include: `fetch_all.py`, `common.py`, `fetch_genes.py`, `fetch_interactions.py`, `fetch_orthologs.py`, `fetch_paralogs.py`, `fetch_allele_detail.py`, `fetch_disease_annotations.py`, `fetch_disease_models.py`, `fetch_phenotypes.py`.

If `fetch_all.py` is missing, the branch override didn't take effect — go back to Check 1.2.

Step 2 — Get a shell inside the candidate image

For ad-hoc inspection, run an interactive container without starting the build:

```
docker compose run --rm alliancemine-builder bash
```

The entrypoint dispatches `bash` to a shell after running the standard config setup (RDS connection check, property templating, etc.).

You'll see roughly:

```
==========================================
AllianceMine Build Container
==========================================
Already compiled, skipping.
Alliance Release: 9.0.0 (from FMS API next)
...
PostgreSQL is ready!
Mode: SHELL
root@xxxxxx:/root#
```

If RDS isn't reachable from inside the container, double-check VPN / security groups before trying any of the next steps.

To leave the shell: `exit`. Because we used `--rm`, the container is removed automatically.

Check 2.1 — Python deps for the fetchers

```
# inside the container
python3 -c "import requests, json, sqlite3" && echo "deps OK"
```

Expected: `deps OK`.

Check 2.2 — fetch_all.py CLI works

```
# inside the container
python3 /root/alliancemine-bio-sources/scripts/fetch_all.py --help | head -20
```

Expected: usage text mentioning `--only`, `--skip`, `--limit`, `--out-dir`.

Step 3 — Smoke-test fetchers in isolation

Inside the same container shell from Step 2, run the orchestrator against a tiny slice (10 IDs per fetcher) into a throwaway directory:

```
mkdir -p /tmp/api-smoke
ALLIANCE_FETCH_CACHE=/tmp/api-smoke-cache \
  python3 /root/alliancemine-bio-sources/scripts/fetch_all.py \
    --limit 10 \
    --out-dir /tmp/api-smoke \
    --verbose
```

Wall time: under 60 seconds.

Check 3.1 — All fetchers exited 0

Look at the `=== Summary ===` block at the end. Every line should end with `ok`. Example:

```
=== Summary ===
  genes              ok  3.2s
  interactions       ok  4.1s
  orthologs          ok  2.8s
  paralogs           ok  1.9s
  allele_detail      ok  2.0s
  disease_annotations  ok  3.5s
  disease_models     ok  2.7s
  phenotypes         ok  2.4s
```

Check 3.2 — Nine TSVs in the output dir

```
ls -la /tmp/api-smoke/
```

Expected files: `alliance-genes.tsv`, `genetic-interactions.tsv`, `molecular-interactions.tsv`, `orthologs.tsv`, `paralogs.tsv`, `allele-detail.tsv`, `disease-annotations-detail.tsv`, `disease-models.tsv`, `phenotypes.tsv`.

> Eight fetchers produce nine TSVs because `fetch_interactions.py` emits both `genetic-interactions.tsv` and `molecular-interactions.tsv`.

Spot-check a file isn't empty:

```
wc -l /tmp/api-smoke/alliance-genes.tsv
head -2 /tmp/api-smoke/alliance-genes.tsv
```

Expected: more than 1 line (header + at least a few data rows), tab- separated columns.

Check 3.3 — Cache populated

```
ls -la /tmp/api-smoke-cache/
```

Expected: one `.sqlite` file per fetcher (`genes.sqlite`, `interactions.sqlite`, etc.). Sizes will be small (KB range) for the `--limit 10` smoke test.

Check 3.4 — Cache hits on rerun

Run the same command again:

```
ALLIANCE_FETCH_CACHE=/tmp/api-smoke-cache \
  python3 /root/alliancemine-bio-sources/scripts/fetch_all.py \
    --limit 10 --out-dir /tmp/api-smoke --verbose
```

Now under 5 seconds total. The verbose log should mention `cache hit` lines from each fetcher.

If you see no cache files but cache hits ≈ 0 on the second run, the `ALLIANCE_FETCH_CACHE` env var isn't being honored — read `/root/alliancemine-bio-sources/scripts/common.py:30` and check the default path the fetcher fell back to.

Cleanup before exiting

```
rm -rf /tmp/api-smoke /tmp/api-smoke-cache
exit
```

Step 4 — Run extract_data with –skip-fms (API only)

Now we're testing the integration code in `docker/alliancemine/scripts/extract_data.py`, not the orchestrator directly.

```
cd ~/agr_intermine_builder/docker/alliancemine
docker compose run --rm alliancemine-builder extract --skip-fms 2>&1 \
  | tee /tmp/extract-api-only.log
```

Wall time: ~20 minutes cold, ~2 minutes warm.

The container will:

1. Skip the S3 sync and FMS download (per `--skip-fms`)
2. Run `fetch_all.py --out-dir /root/data/api/` with `ALLIANCE_FETCH_CACHE=/root/data/api-cache`

Check 4.1 — Exit was clean

Last log line should be:

```
FMS: 0 succeeded, 0 failed, 0 not in snapshot, out of 49. API fetch: ok.
```

Plus exit code:

```
echo $?
# Expect: 0
```

Check 4.2 — Nine TSVs land on the host

`./data/` is bind-mounted, so the outputs are visible from the host:

```
ls -la ./data/api/
```

Expected: nine TSVs (same names as Check 3.2), this time at full size (MB range — `alliance-genes.tsv` and the interaction files are usually the biggest).

Check 4.3 — Cache populated

```
ls -la ./data/api-cache/
du -sh ./data/api-cache/
```

Expected: nine `.sqlite` files, total size in the tens-to-hundreds of MB range depending on which MODs got fetched.

Check 4.4 — Re-run hits the cache

```
docker compose run --rm alliancemine-builder extract --skip-fms 2>&1 \
  | tail -30
```

Should finish in under 5 minutes. Look for `cache hits >> cache misses` in the per-fetcher summary lines.

Step 5 — Run extract_data end-to-end (FMS + API)

Once Step 4 looks good, exercise the full stage including FMS:

```
docker compose run --rm alliancemine-builder extract 2>&1 \
  | tee /tmp/extract-full.log
```

Wall time: ~25 minutes cold (5 min FMS + 20 min API), ~7 min warm (5 min FMS + 2 min API).

Check 5.1 — All three passes succeeded

Tail of the log should show:

```
FMS: 49 succeeded, 0 failed, 0 not in snapshot, out of 49. API fetch: ok.
```

Check 5.2 — All data dirs populated

```
ls ./data/intermine/ | wc -l    # S3 sync — expect dozens of files
ls ./data/fms/ | wc -l          # FMS — expect ~40 files
ls ./data/genes/ | wc -l        # FMS BGI — expect ~9 files
ls ./data/api/ | wc -l          # API fetchers — expect 9
```

Step 6 — Run the full build

This is the actual test of whether the new TSVs integrate cleanly with the bio-source modules and the rest of the project.

Use a unique RC number so the candidate build doesn't collide with any in-flight production builds on RDS. The build will:

- Create the candidate database `alliancemine_9_0_0_rc99` on RDS (if missing)
- Create candidate Solr cores `alliancemine-search-9.0.0-rc99` and `alliancemine-autocomplete-9.0.0-rc99` on multitenant (preflight, idempotent)
- Run the 6-stage pipeline end to end

```
tmux new-session -s api-test
cd ~/agr_intermine_builder/docker/alliancemine
docker compose run --rm -e RC_NUMBER=99 alliancemine-builder build 2>&1 \
  | tee /tmp/build-9.0.0-api.log
```

Wall time: 5-8 hours.

Detach tmux: `Ctrl+B D`. Reattach: `tmux attach -t api-test`.

Monitoring while it runs

In another shell on the same host:

```
# Tail the log
tail -f /tmp/build-9.0.0-api.log

# Find the running container name
docker ps --format '{{.Names}}' | grep builder

# Get a shell inside the running container (read-only inspection)
CNAME=$(docker ps --format '{{.Names}}' | grep builder)
docker exec -it $CNAME bash

# Inside: check progress via project_build state files
ls -la /root/alliancemine/build/
```

Check 6.1 — extract_data stage passed

After roughly 25 minutes, the log shows:

```
STAGE 2/6: Extracting Alliance Data (S3 + FMS + API)
...
[STAGE COMPLETE]
```

If it failed here, the integration broke — investigate `/tmp/build-9.0.0-api.log` and skip the rest.

Check 6.2 — project_build accepts the new sources

Search the log for the new bio-source modules:

```
grep -E 'alliance-paralogs|alliance-disease-models|alliance-phenotypes' \
  /tmp/build-9.0.0-api.log | head
```

Each should appear with a `[INTEGRATING]` and `[FINISHED]` marker, no errors in between.

Check 6.3 — Data landed in the candidate database

After the build completes:

```
PGPASSWORD='<rds-password>' psql \
  -h intermine-postgres.cmnnhlso7wdi.us-east-1.rds.amazonaws.com \
  -U postgres \
  -d alliancemine_9_0_0_rc99 \
  -c "SELECT COUNT(*) FROM paralogue;
      SELECT COUNT(*) FROM phenotypeannotation;
      SELECT COUNT(*) FROM diseasemodel;"
```

Expected: non-zero counts for each. Compare to the previous build's counts if you have them — anything within ~10% is normal release-to- release drift; a 10x change probably means something went wrong.

Step 7 — Promote the candidate

If Step 6 finished cleanly:

```
# Tag the candidate as the new production image
docker tag alliancemine-builder:9.0.0-api alliancemine-builder:latest

# Verify both tags point at the same image ID
docker images alliancemine-builder
```

Expected: two rows (`9.0.0-api` and `latest`) with identical IMAGE IDs.

Update `.env` to drop the override so future builds use the production image:

```
sed -i.bak '/^IMAGE_TAG=9.0.0-api$/d' .env
sed -i.bak '/^BIO_SOURCES_BRANCH=wire-api-sources$/d' .env
rm .env.bak
```

The next regular `docker compose run --rm alliancemine-builder build` will use `alliancemine-builder:latest` — which is now the validated candidate.

Note: The candidate build's database (`alliancemine_9_0_0_rc99`) and Solr cores (`alliancemine-search-9.0.0-rc99` etc.) still exist after promotion — they're independent of the image promotion. Decide whether to: - **Keep them** as a safety net while the production build catches up, or - **Drop them** to free RDS storage / Solr disk: `bash # DB PGPASSWORD='...' psql -h ... -d postgres -c \ 'DROP DATABASE "alliancemine_9_0_0_rc99"' # Solr cores (from multitenant host) sudo rm -rf /var/solr/data/alliancemine-search-9.0.0-rc99 \ /var/solr/data/alliancemine-autocomplete-9.0.0-rc99 sudo -u solr /opt/solr/bin/solr restart`

Useful container commands cheat sheet

Action	Command
Build candidate from current `.env`	`docker compose build --no-cache alliancemine-builder`
Interactive shell, no build	`docker compose run --rm alliancemine-builder bash`
Run a single stage	`docker compose run --rm alliancemine-builder extract --skip-fms`
Shell into a running build	`docker exec -it $(docker ps --format '{{.Names}}' | grep builder) bash`
Tail the build log	`tail -f /tmp/build-9.0.0-api.log`
Stop a running build cleanly	`docker stop $(docker ps --format '{{.Names}}' | grep builder)`
List candidate vs production tags	`docker images alliancemine-builder`
Wipe data dir between tests	`rm -rf docker/alliancemine/data/{api,api-cache}`
Drop the candidate RC database	`psql -h ... -d postgres -c 'DROP DATABASE "alliancemine_9_0_0_rc99"'`

Troubleshooting

"API fetcher orchestrator not found"

The image was built without the `wire-api-sources` branch (or before the branch existed). Re-do Step 1 with `--no-cache` and confirm `BIO_SOURCES_BRANCH=wire-api-sources` is in `.env`.

A single fetcher failed but the rest succeeded

`fetch_all.py` exits non-zero when any fetcher fails. The summary block names the failed one. Re-run just that fetcher to inspect:

```
docker compose run --rm alliancemine-builder bash
# inside:
python3 /root/alliancemine-bio-sources/scripts/fetch_phenotypes.py \
  --out-dir /root/data/api --verbose --limit 50
```

Most failures are transient API timeouts; the cache is preserved so the rerun starts with most work already done.

Cache fills the disk

Each `.sqlite` file caches every URL response. For multi-MOD builds the cache can grow to several GB. Wipe it if you want a forced cold run:

```
rm -rf docker/alliancemine/data/api-cache/
```

Want to test fetcher edits without a rebuild

Bind-mount the host clone over the image's clone for the duration of one run:

```
docker compose run --rm \
  -v $(pwd)/../../alliancemine-bio-sources:/root/alliancemine-bio-sources:ro \
  alliancemine-builder extract --skip-fms
```

Useful only for iterating on Python; for a real validation, rebuild the image so the test reflects exactly what would ship.